Course Syllabus

Course Title:
C Programming & User-Space Rootkit Techniques (Educational Red Teaming)

Course Length:
15 Weeks (1 Semester)

Course Type:
Independent Study / Project-Based Lab

Prerequisites:

- Basic Linux command line usage
- No prior C programming required

Platform:

- Linux (local machine only)
- No network-based attacks
- User-space only (no kernel modules)

Course Description

This course is a **project-driven introduction to the C programming language through the lens of defensive red-team cybersecurity**. Students will incrementally build a **user-space rootkit simulation** using `LD_PRELOAD`, while simultaneously learning core and advanced C concepts.

The course emphasizes:

- Low-level systems programming
- Dynamic linking and ELF internals
- Realistic attacker techniques used in user-space malware
- Defensive detection, logging, and mitigation strategies

All work is conducted in a **controlled, ethical lab environment**, with equal emphasis on **how attacks work and how they are detected**.

Course Learning Objectives

By the end of the semester, the student will be able to:

1. Write **safe, structured C programs** using pointers, structs, and dynamic memory
2. Understand **Linux process execution, shared libraries, and dynamic linking**
3. Implement **user-space function hooking** using `LD_PRELOAD`

4. Simulate common **rootkit behaviors** (file hiding, process hiding, credential interception)
5. Analyze and document **blue-team detection techniques**
6. Explain **why rootkit techniques work and why they fail**
7. Produce a **professional technical report** with ethical framing

---

Final Project Goal

Deliverable:
A modular **user-space rootkit simulation framework** written in C that demonstrates multiple red-team techniques using `LD_PRELOAD`, accompanied by a **defensive analysis report**.

Project Artifacts:

- Source code (well-documented)
- Test binaries
- Lab notes per module
- Final written report (10–15 pages)

Weekly Schedule & Milestones (With Learning Resources)

Week 1 — Linux & C Foundations

C Topics

- Compiling with `gcc`
- `main()`, headers, return values
- Basic I/O (`printf`, `puts`)
- Makefiles (intro)

Learn From

- *The C Programming Language* (K&R), Ch. 1
- https://www.learn-c.org
- https://linuxjourney.com (Linux basics)

Security Focus

- What rootkits are (user-space vs kernel)
- Ethical boundaries

Milestone

- Build and run simple C programs
- Write a short threat-model overview

Defense

- Why most malware relies on programming mistakes

Week 2 — Memory & Pointers (Critical Week)

C Topics

- Stack vs heap
- Pointers and dereferencing
- `malloc`, `free`
- Common memory bugs

Learn From

- K&R Ch. 5
- https://beej.us/guide/bgc/ (Beej's Guide to C)
- https://exploit.education (conceptual memory errors)

Security Focus

- Why C is used in malware
- Memory corruption risks

Milestone

- Write programs using dynamic memory safely

Defense

- AddressSanitizer
- Compiler warnings (`-Wall -Wextra`)

Week 3 — Strings, Buffers, and libc

C Topics

- `char *`, arrays
- `strlen`, `strcpy`, `strcmp`
- Safe vs unsafe functions

Learn From

- Beej's Guide to C (Strings section)
- `man string`, `man strcpy`
- https://cdecl.org (pointer clarity)

Security Focus

- Buffer overflows (conceptual)
- Why libc is a prime target

Milestone

- Write a string-processing utility

Defense

- `strace`
- Hardened libc behavior

Week 4 — Linux Processes & syscalls

C Topics

- `fork`, `exec`
- `getpid`, `getuid`
- Return codes

Learn From

- Beej's Guide to Unix IPC
- `man fork`, `man execve`
- https://man7.org/linux/man-pages/

Security Focus

- How programs interact with the OS
- Why attackers hook functions

Milestone

- Build a simple process inspector

Defense

- Process auditing basics

Week 5 — Shared Libraries & ELF Basics

C Topics

- `.so` files
- Position-independent code
- `ldd`

Learn From

- https://tldp.org/HOWTO/Program-Library-HOWTO/
- `man ld.so`, `man ldd`
- https://lief.quarkslab.com/doc/latest/

Security Focus

- ELF loading process
- Dynamic linking attack surface

Milestone

- Create and load a shared library

Defense

- Static linking vs dynamic linking

Week 6 — `LD_PRELOAD` Fundamentals

C Topics

- Function overriding
- Symbol resolution
- `dlsym`, `RTLD_NEXT`

Learn From

- `man ld.so`
- https://github.com/poliva/ldpreloadhook
- https://catonmat.net/ld-preload

Security Focus

- User-space hooking

Milestone

- Hook `puts()` or `printf()`

Defense

- Detecting `LD_PRELOAD`
- Environment inspection

Week 7 — File Hiding Simulation

C Topics

- Structs
- Hooking `readdir()`

Learn From

- `man readdir`
- https://github.com/mfontanini/Programs-Scripts/tree/master/rootkits
- Linux `/proc` documentation

Security Focus

- Rootkit file hiding behavior

Milestone

- Hide files with a chosen prefix

Defense

- Comparing raw syscalls vs libc
- Integrity checks

Week 8 — Process Hiding Simulation

C Topics

- Parsing `/proc`
- Advanced `readdir()` hooks

Learn From

- https://man7.org/linux/man-pages/man5/proc.5.html
- ps/top source code (reading only)

Security Focus

- Process invisibility

Milestone

- Hide a test process from `ps`

Defense

- Cross-tool verification (`ps`, `/proc`, `top`)

Week 9 — Credential Interception (Educational)

C Topics

- Hooking `read()`
- Buffer inspection

Learn From

- `man read`
- https://www.win.tue.nl/~aeb/linux/lk/lk-10.html

Security Focus

- Password interception theory

Milestone

- Log mock credentials from a test program

Defense

- PAM hardening
- Audit logs

Week 10 — Persistence Techniques

C Topics

- Environment variables
- Shell startup files

Learn From

- `man environ`
- Bash startup documentation
- https://attack.mitre.org (Persistence concepts)

Security Focus

- User-space persistence

Milestone

- Simulate persistence safely

Defense

- Startup audits
- Environment sanitization

Week 11 — Evasion & Anti-Analysis

C Topics

- Conditional logic
- Timing checks

Learn From

- `man ptrace`
- https://anti-debug.checkpoint.com
- Debugger detection writeups

Security Focus

- Anti-debugging basics

Milestone

- Disable hooks when debugger detected

Defense

- Behavioral detection

Week 12 — Rootkit Architecture & Cleanup

C Topics

- Modular design
- Error handling

Learn From

- Clean Code principles (C-applicable sections)
- https://github.com/torvalds/linux (style inspiration)

Security Focus

- Stealth vs stability tradeoffs

Milestone

- Combine modules into one framework

Week 13 — Defensive Testing

Focus

- `strace`, `ltrace`
- Environment diffing

Learn From

- `man strace`, `man ltrace`
- https://www.brendangregg.com/linuxperf.html

Milestone

- Detect your own rootkit

Week 14 — Documentation & Ethics

Focus

- Writing defensive reports
- Ethical disclosure

Learn From

- MITRE ATT&CK documentation style
- Academic security papers (format)

Milestone

- Draft final report

Week 15 — Final Submission

Deliverables

- Source code
- Final report
- Architecture diagram
- Reflection on defense vs offense

Grading Criteria (Suggested)

- Code correctness & safety — 30%
- Technical depth — 25%
- Defensive analysis — 25%
- Documentation & ethics — 20%

Ethical Statement

This project is conducted **solely for educational and defensive learning purposes**. No techniques are deployed outside a controlled lab. The goal is to **understand attacker behavior in order to improve detection and defense**.